

節 4 / 共 4 節

如何驗證 LLM 的產物

我憑什麼相信它的輸出？

編譯器給的，就是那個缺掉的回饋訊號

回接上午 3-D 的論點。

「動態語言在沒有測試的情況下，
agent 寫完你根本不知道對不對。」

- Rust 的編譯器、TypeScript 的 `tsc` 就是這個訊號
- 這一節要替 LLM 的輸出親手做一個

重點 本節產出是一張圖。那張圖就是這門課的結論。

我們不問「它對不對」。

我們問「它多常對」。

昨天說過的那件事：同樣輸入不保證同樣輸出。一次成功不算數。

4-A

四層防線

由弱到強，由便宜到貴。

四層防線

層	檢查什麼	怎麼做	成本
1 格式	是不是合法的 JSON	<code>JSON.parse()</code> 成不成功	幾乎 0
2 Schema	欄位在不在、型別對不對	zod / pydantic	低
3 邏輯	欄位之間的關係合不合理	自己寫規則	中
4 語意	內容是不是真的對	人工，或另一個模型	高

重點 大部分人只做到第 2 層就停了。而 LLM 最常出錯的地方在第 3 層。

一個真的例子：一封訂購信

這就是下午 Lab 7 的第一封信。四層防線在它身上分別抓到什麼？

- 藍色原子筆，一打（12 支），
單價 25 元
- A4 影印紙 3 包，一包 145 元

系學會有簽約折扣 50 元。

付款用新台幣。

訂單日期 2026-09-01，

希望 9/5 前出貨。

單號 P0-20260901-014。

emails/01-basic.txt

```
{  
  "order_id": "P0-20260901-014",  
  "currency": "TWD",  
  "items": [  
    {"name": "藍色原子筆",  
      "unit_price": 25, "qty": 12},  
    {"name": "A4 影印紙",  
      "unit_price": 145, "qty": 3}  
  ],  
  "subtotal": 735,  
  "discount": 50,  
  "total": 735  
}
```

模型抽出來的 JSON

注意 每一個欄位單獨看都是對的。品項對、單價對、幣別對、subtotal 也對。

四層各攔下什麼

同一份輸出，走一遍四層防線。

層	結果	為什麼
1 格式	✓	<code>JSON.parse()</code> 過了。是合法的 JSON
2 Schema	✓	欄位齊、型別對、currency 在允許清單裡 —— 完全通過
3 邏輯	✗	total 是 735，但 $\text{subtotal}(735) - \text{discount}(50) = 685$
4 語意	—	這題不需要。「摘要有沒有反映原文」那種才要

重點 **折扣被吃掉了**。而前兩層**一個字都沒抱怨** —— 這筆資料會安安靜靜地進你的資料庫，直到有人對帳。

實務 Lab 7 的 `validate.ts` 就是這樣寫的：`Math.abs(subtotal - discount - total) > 0.001`。一行。

第 1 層：格式

- 最便宜、最無腦，但擋掉的問題最多
- 常見的失敗：多包了一層 markdown code fence、前面加一句「好的，以下是您要的 JSON：」

```
try {  
  const data = JSON.parse(output)  
} catch {  
  // 這一層就擋下來了，連看內容都不用  
}
```

實務 對策很簡單：prompt 裡明確寫「**只輸出 JSON，不要 markdown code fence，不要任何解釋**」。有些 API 還提供強制 JSON 輸出的參數，能用就用。

第 2 層：Schema

- 欄位在不在？型別對不對？必填的有沒有漏？
- 用現成工具，不要自己寫 if：**zod** (TS)、**pydantic** (Python)
- Lab 7 為了零依賴手寫了這一層 —— 正式專案請直接用 zod

```
const Order = z.object({
  order_id: z.string(),
  currency: z.enum(["TWD", "USD", "JPY"]),
  items: z.array(Item).min(1),
  total: z.number().positive(),
})
```

重點 寫完 schema 有個附加好處：**你可以把它直接貼進 prompt 裡**。給 schema 遠比用文字描述欄位有效。

第 3 層：邏輯 —— 這一節的重點

每個欄位**單獨看都很合理**，
合起來卻是錯的。

- 這是 LLM 最常出錯、也最容易被忽略的一層
- 因為 schema 驗證會**完全通過**，錯誤靜靜地流進你的資料庫
- 沒有任何現成工具能幫你 —— **這些規則只有你知道**

邏輯層的規則長這樣

規則	為什麼 LLM 會錯
<code>start_date < end_date</code>	它從文字抽日期時，很容易抽反
<code>total == sum(items.price × qty)</code>	它不會算數 。它是在「產生一個看起來合理的數字」
<code>status == "shipped" → 必須有 tracking_no</code>	它不知道你的業務規則
<code>currency</code> 必須在允許清單內	它會自己發明 NT\$、NTD、TWD 三種寫法
<code>discount <= subtotal</code>	它沒有「不合理」的概念

注意 第二條是重災區。**LLM 算數學是用猜的**，不是用算的。任何需要計算的欄位，都應該由你的程式算，不要問它。

第 4 層：語意

- 「這段摘要有沒有正確反映原文？」 「這個分類對不對？」
- 只能靠**人工**，或者**另一個模型**來判斷
- 兩種都很貴：人工貴在時間，模型貴在錢，而且**裁判本身也會出錯**

重點 策略是：**能往前三層推的，全部往前推**。設計你的輸出格式時，就刻意讓它變成「可以用程式檢查」的形狀。

實務 例如：與其要它「寫一段評語」，不如要它「從 5 個預設標籤中選 1 個 + 引用原文的哪一句」。後者可以用程式驗證，前者不行。

四個強化手段，由效果大到小

1. **給 schema，不要給文字描述** —— 直接把 zod / JSON Schema 貼進去
2. **給例子，而且要給反例** —— 「以下是錯誤示範，因為 total 沒對上」
3. **明確禁止多餘輸出** —— 「只輸出 JSON，不要 code fence，不要解釋」
4. **驗證失敗就把錯誤訊息餵回去重試** —— 回到 loop engineering

重點 第 2 點的效果最出乎意料：**一個反例，抵得過三段規則描述。**

反例長什麼樣

以下是「錯誤」的輸出，不要這樣做：

```
{ "items": [{"price": 100, "qty": 2}], "total": 300 }
```

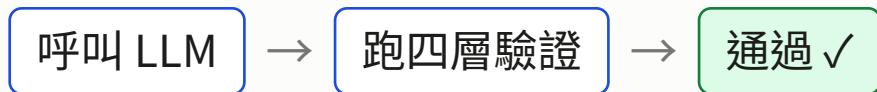
錯在哪裡：total 應該是 $100 \times 2 = 200$ ，不是 300。

每一次輸出前，請自己重新計算一次 total。

- 反例讓模型知道「錯」長什麼樣子，而不只是「對」長什麼樣子
- 最好的反例來源：你上一輪跑出來的真實失敗案例

實務 這就是 Lab 7 的做法 —— 用第一輪的失敗，餵養第二輪的 prompt。

重試迴路



↑ 失敗時：把**具體的錯誤訊息**接在原本的 prompt 後面，重來（最多 3 次）

注意 關鍵在「**具體的**」。「格式錯誤」沒有用；「total 是 300，但 items 加起來是 200」才有用。

重點 這跟昨天 Lab 4 的 `loop.ts` 是**完全一樣**的機制，只是換了個場景。

這整套東西業界叫 eval

它叫 **eval**。

- 用一組**固定的測資**跑 N 次、用程式判斷對錯、統計通過率
- 每一家做 LLM 產品的公司都有一套自己的 eval
- Lab 7 產出的那張圖，就是一個最小的 **eval harness**

實務 面試講「我會用 AI」沒有用；講「我做過 eval，這是失敗率下降的曲線」是另一個量級。

4-B

Lab 7：輸出驗證器

用數據證明 prompt 改對了。

Lab 7 輸出驗證器

60 分鐘

目標 用數據證明「改 prompt 之後，失敗率真的下降了」。

題目：從一封自由格式的訂購 email，抽出結構化的訂單 JSON。

1. **基準** 用最爛的 prompt 跑 **10** 次，用 validator 統計失敗率與**失敗類型**
2. **強化** 加上 schema + 反例，再跑 10 次
3. **迴路** 加上「失敗就把錯誤訊息餵回去重試」，再跑 10 次
4. 畫出「prompt 版本 vs 失敗率」的變化

實務 基礎驗收 你有一張三個版本的失敗率對照，而且能解釋為什麼。

重點 進階 自己補寫邏輯層規則（至少 2 條 validator 沒寫的）。

`node run.ts --version v1 --n 10`（再跑 v2、v3）→ `node report.ts`

Lab 7 記錄表

統計腳本與繪圖腳本已經提供，你只需要改 prompt 字串。

版本	通過	失敗	最常見的失敗類型	平均 tokens
v1 爛 prompt	／10			
v2 + schema + 反例	／10			
v3 + 重試迴路	／10			

重點 最右邊那一欄別跳過。乘上模型單價，就是上午講的「這個功能一個月要花多少錢」。

實務 「失敗類型」比「失敗率」重要。10 次的樣本很小，數字會抖；但失敗的**種類**很穩定，而且那才是你能動手修的東西。

如果你的失敗率沒有下降

那也是一個有效的結果。

- 10 次的樣本，統計波動很大。改了 prompt 反而變差，是正常的
- 免費模型的雜訊又更大
- **替代驗收**：你能分類出 3 種不同的失敗類型，並解釋每種為什麼會發生

注意 這一頁本身就是一堂課：**真實的 eval 就是這樣，很多改動沒有效果，而你得靠數據才知道。**憑感覺調 prompt 的人，永遠不知道自己是不是在原地打轉。

4-C

為什麼不要全部 vibe coding

一個我自己完全用 AI 寫出來的專案，以及它長歪的地方。

先講結論的另一半：它真的做得出來

wordforge —— 一個語言學習的桌面應用程式。Rust + Tauri，全程 vibe coding。

規模	91 個 commit，8 個 crate，3 萬多行 Rust
功能	多語言字典匯入、間隔重複、LLM 出題（閱讀測驗/克漏字/翻譯/文法）、TTS
做法	每次就是跟 agent 說「我要加一個 X」，然後它就加了

重點 速度是真的。這種規模的東西，以前我不會在業餘時間開始做。所以接下來要講的問題，不是「所以不要用 AI」。

症狀 1：同一個地方，一直回來改

從 git log 直接看得出來的東西：

「生詞候選」這件事	六個 commit 的區間裡改了 4 次
「複習紀錄」這件事	連續 3 個 commit 各補一次

- 每一次都很合理：發現一個 case 沒處理好 → 跟 agent 說 → 它補上 → 能動了
- 問題是**第四次的時候，前三次的理由已經沒有人記得了**—— 包括我，也包括它

注意 這是最容易被忽略的症狀，因為**每一步看起來都是進步**。只有把 log 拉開來看，才會發現你在原地繞。

症狀 2：檔案長到被迫切開

檔案	被改過幾次	後來怎麼了
前端的 <code>api.ts</code>	30 次	切成 14 個檔
出題引擎 <code>engine.rs</code>	30 次	切成 7 個檔

- 切開的那個 commit 訊息寫的是：「…**並把大檔案切開**」
- 注意那個「並」字 —— 它是**順便**做的，不是規劃好的

重點 重構變成了被迫的維修，而不是排進行程的工作。等到你「順便」想切的時候，通常已經有點晚了。

症狀 3：同一個概念，長出好幾個版本

這是最貴的一個。專案裡有四個「跟模型說它做錯了」的函式：

<code>format_retry</code>	格式壞掉
<code>option_notes_retry</code>	逐選項解說沒生齊
<code>few_new_words_retry</code>	太簡單，該用的字沒用夠
<code>coverage_retry</code>	太難，難字要換掉

- 四個函式、**各自不同的參數**，呼叫點散在**兩個檔案共 6 處**
- 每一個都是某次「這個情況也要重試」的時候長出來的

而同一件事的另一半，驗證那一側，早就收攏成一個統一的 Problem 型別了。

把兩邊的程式碼並排看

同一個專案，同一件事的兩半。

沒收斂的那一半

```
fn format_retry(
    problems, previous)

fn option_notes_retry(
    native_lang, field, missing, prev)

fn few_new_words_retry(
    actual_coverage, target, unused, prev)

fn coverage_retry(
    actual_coverage, target, offenders, prev)
```

prompts.rs 四個函式、四種參數，呼叫點散在兩檔共 6 處

收斂了的那一半

```
/// 一處驗收沒過的地方。
pub struct Problem {
    pub path: String,
    pub detail: String,
}
```

validate.rs 一個型別，全部共用

不管哪一種驗收失敗，產出的都是同一個 Problem。

注意 加第五種重試 → **再寫一個函式 + 一個呼叫點**；加第五種檢查 → **多回傳一個 Problem**。

症狀 4：它會幫你切檔案，但切得很粗糙

而且切完之後，你以為問題解決了。

我叫它把 <code>engine.rs</code> 切開	切完之後
30 次修改 之後終於受不了	變成 <code>engine/</code> 底下 7 個檔
看起來像是解決了	但最大的 <code>grade.rs</code> 還是 1071 行

整個專案 77 個 `.rs`、中位數 267 行 —— 但**其中 24 個 (31%) 裝了 69% 的程式碼**。

注意 它照「行數太多」在切，不是照「這裡是一個概念」在切 —— 大檔只變成幾個中大檔。代價在後面：**要改的時候，它很難定位到該改哪一段。**

判斷「它錯了」，收攏成一塊了。

講「你錯在哪」，沒有。

同一個概念的兩半，一半收斂了，一半沒有。這就是「邏輯不一樣」的實際長相。

它不會問「專案裡是不是已經有做法了」

需求是「加一個功能」，產出是「一個能動的功能」。

中間沒有人問「這件事已經有做法了嗎」。

- 三週前的決定不在 context 裡
- 它傾向**在現有結構旁邊加一個新的**，那樣最不會弄壞既有的
- 單次都正確，**錯的是加起來的樣子**

注意 這不是模型不夠聰明。換更強的模型，一樣不知道三週前的決定。

問題不是它寫不出來，是沒人在管形狀

不是它**寫不出來**，
是沒有人在管**這些東西加起來是什麼形狀**。

- 傳統開發裡管形狀的人是你，因為你被迫每天面對整份程式碼
- vibe coding 把你從「每天面對」變成「每次驗收」
- **於是形狀就沒人看了**

重點 要補的不是 AI 的能力，是**開發節奏**。四個做法：

而且形狀壞掉之後，你是用 token 在付錢

這是很多人沒算過的一筆帳。

	有人在管形狀	沒人管
要改一個功能	讀那個 200 行的模組	讀一個 1000 行、混了三件事的檔
它找得到嗎	檔名就講完了	得先搜尋、猜、讀好幾個檔
每次改的 context	小而且穩定	越來越大
改壞的機率	低	高 —— 因為它沒看到全部

重點 長期維護一個專案，「理解每個部分在做什麼」不是潔癖，是成本控制。形狀壞掉的專案，同樣一個需求要燒掉好幾倍的 token，而且更容易改壞。

實務 這也是為什麼今天上午講的那三句話，回到這裡還是同一件事：你給它的 context 品質，決定它的產出品質。

做法 1：把規矩持久化，而且把事故寫進去

- 這就是節 2 的 AGENTS.md。但真正有用的不是「請寫好程式碼」那種話
- wordforge 的那份寫了 268 行，其中有一節標題是：

「已經發生過的實例」

- 底下用表格列了 **8 條真的發生過** 的事故，每條就寫「寫死了什麼 → 後果」

重點 **事故比規則有效**。「不要寫死」它會忘記；「上次寫死 en 造成什麼後果」它會照著避開。

實務 下一頁就是那八條的原文。

那八條長什麼樣

.claude/CLAUDE.md 裡「已經發生過的實例」整張表，原封不動。

曾經寫死了什麼	後果
各處的 "en"	「換字典就能學別的語言」名存實亡
覆蓋率目標 0.96	使用者想調 90% 法則的門檻，調不了
gpt-5.6-luna 當 codex 預設	在 codex-cli 0.142.5 上直接被拒絕
前後端各一份模型清單	兩邊長歪，而且都會過期
「太難」用寫死的 90% 難度帶	目標設 98% 時那個判斷完全不生效
生詞選詞完全決定性	每篇文章都拿到一模一樣的六個字
GRAMMAR_POINTS 一份英文清單	學日文的人拿到英文文法術語
文法選單只列 <code>state != null</code>	沒選過的點就永遠練不到

注意 每一條都**不是打字錯誤**，是當下「先讓功能動起來」的合理決定——而那正是這類 bug 的來源。

做法 2：把「整合」排進節奏，不要等它痛

- 每加 N 個功能，就排一次**不加功能的 commit**：只把散開的東西收攏
- 收攏的對象很好找 —— **看哪個檔案最近被改最多次**

問自己	動作
這件事在專案裡有幾種做法？	超過一種就收成一種
這個檔案這個月被改幾次？	次數高的地方通常是抽象抓錯了
有沒有「第 N 個 XXX_retry」？	第三個出現時就該停下來收

實務 最後一條是我的教訓：**第二個還可以忍，第三個就是訊號。**

做法 3：加功能之前，先讓它回答一個問題

「動手之前，先找出專案裡
已經有沒有類似的做法，
有的話沿用，不要另外開一套。」

- 這一句寫進 AGENTS.md，成本是一行，效果是四個 retry 變成一個
- 它其實**做得到**，只是你沒叫它做 —— 它有讀檔工具，它可以自己去找

重點 這是全場投報率最高的一句話。**今天回去就可以加。**

做法 4：先有測試，才敢整合

- 整合最大的阻力是「我不敢動，動了不知道會壞什麼」
- wordforge 裡最大的檔案就是測試檔：3842 行、73 個測試，涵蓋「出題 → 作答 → 批改 → 進牌組」整條路
- （全專案 520 個測試，這一個檔佔了最關鍵的那條路）
- 而且那些測試用的是假的 LLM。它的開頭註解寫著：

「真正要驗證的是編排邏輯，不是模型本身。
用真模型反而讓測試不穩定又慢。」

重點 這正好呼應今天上半場：你要的是一個穩定、免費、快的回饋訊號。真模型不穩定，所以測試裡不要用它。

Vibe coding 讓你寫得快。

它不會讓你改得快。

而軟體的一生，大部分時間都在改。

換你想一想

20 分鐘，跟旁邊的人討論，然後我們聊幾個。

如果你手上有一個自己的專案（或今天 Lab 1 做的那個）：

1. 有沒有哪個概念，你已經**做了不只一次**？
2. 哪個檔案是你最常回去改的？
3. 如果現在要寫一條「事故規則」進 AGENTS.md，你會寫什麼？

實務 沒有自己專案的人，就看 Lab 1 的檢核表——**第 12 列「用了 date.ts 的 helper」**，那就是這個問題的種子——**整張表的第 9~14 項（風格層）**，每一項都是「每次都一樣」的規矩。

4-D

工程師現狀

誠實地講，但不販賣焦慮。

什麼真的被壓縮了

- 樣板碼、初階 CRUD、一次性腳本、格式轉換
- 「照著文件把 API 串起來」這類工作
- 寫測試的第一版、寫文件的第一版

共同點：**問題已經被定義清楚了，只剩下打字。**

說實話：這部分佔了很多初階工程師工作的一大半。假裝它沒發生沒有意義。

什麼反而更值錢了

- **把模糊的問題定義清楚** —— 你這兩天做的每一個 lab，一半的工作都是這個
- **系統設計與取捨** —— 節 3 那三句話（key 放哪、誰付錢、誰扛延遲）
- **判斷輸出對不對** —— 也就是這一節
- **資安** —— 因為 agent 有權限，而且會被騙

共同點：**你得知道「對」長什麼樣子。**

「會用 agent」很快就不是差異化了。

差異化在於

你能不能證明它是對的。

這就是為什麼這門課花了最後三小時在講驗證。

四件今天就能做的事

帶走這四條就夠了。

1. 留著 **Lab 7 那張圖** —— 面試時拿出來講，勝過「我熟悉 AI 工具」
2. 今晚寫一份 AGENTS.md —— 十分鐘的事
3. 每次 AI 寫完，養成問一句：**「我要怎麼知道這是對的？」**
4. 不要停在 prompt —— 另外三層在哪裡已經很清楚了

實務 還有一件不用等的事：**去教別人**。這兩天做得快的那幾位，幫同桌時學到最多。

回到第一頁那句話

LLM 只是個會猜下一個字的東西。
讓它變成能交付的系統的，
是你在它外面搭的**回饋迴路**。

節次	你搭了什麼	具體產出
節 1	派工跟問答差在哪	streak 功能 + 檢核表 5/14 → 13/14
節 2	它有什麼工具與規矩	AGENTS.md + 迴路，遵守率明顯上升
節 3	它看得到什麼	RAG 對照、抓到自己的 key
節 4	你怎麼確認它對、怎麼讓它不長歪	失敗率下降的那張圖

課後資源與下一步

自己接著做

- 幫自己的專案寫 AGENTS.md
- 把附錄的 MCP 自習包做完 (labs/appendix-mcp/)
- 試試 subagent extension
- 挑一個你自己的專案，做一次「整合 commit」

這門課沒講但值得看

- 多模態：圖片、語音輸入
- streaming 與延遲的 UX
- 模型選擇：什麼任務用什麼等級

社團的下一步

〔講師填：要不要組隊做一個社團自己的工具？

用今天學到的節奏，而不是一路 vibe 下去〕

實務 社團相對於學分課的唯一優勢，就是**可以把東西真的做完**——而「做完」正是 vibe coding 最容易失守的地方。

謝謝這兩天。

有問題隨時來問。祝你們做出點什麼。